Milestone Two: Enhancement One

Software Design and Engineering — Aaron Jones, CS 499

1. Artifact Description

The artifact is a set of Contact, Task, and Appointment management services, originally built for CS 320: Software Testing and Automation for a fictional client ("Grand Strand Systems"). It consists of three parallel Java modules — a Contact/ContactService pair, a Task/TaskService pair, and an Appointment/AppointmentService pair — each with a validated, immutable-ID entity class, a service class exposing add/delete/update/get operations, and a JUnit test suite covering valid and invalid inputs.

2. Justification for Inclusion

I selected this artifact for the Software Design and Engineering category because it's the clearest demonstration I have of defensive, validated object design paired with a real test suite — every constructor and setter enforces its own invariants (ID length and immutability, field-length limits, a 10-digit phone number, a future-only appointment date) and throws a specific exception on violation rather than failing silently. The code review I completed earlier in this course (Milestone One) also surfaced concrete, fixable problems in it, which made it a good candidate for a genuine before/after enhancement rather than a project that was already in a finished state.

The specific components that showcase my skills in software development are the entity classes' constructor validation, the service classes' add/delete/update logic, and — after this milestone's enhancement — the repository abstraction described below, which is the most direct evidence of applying a real design pattern rather than writing code that only works for one hardcoded scenario.

How the artifact was improved: the code review found that each service owned its storage directly as a private HashMap, which made the services impossible to test against anything other than that one in-memory implementation, and directly caused the gap I identified in my own CS 320 project reflection — no integration testing, because there was no external data source to integrate with. This milestone's enhancement was to introduce a repository interface (findById, save, existsById, deleteById, findAll) for each of the three entities, so ContactService, TaskService, and AppointmentService now depend on that interface instead of a concrete map:

```
// Before: ContactService owned its storage directly
public class ContactService {
    private final Map<String, Contact> contacts = new HashMap<>();
    ...
}

// After: ContactService depends on an injected abstraction
public class ContactService {
    private final ContactRepository repository;
    public ContactService() { this(new InMemoryContactRepository()); }   //
unchanged default behavior
    public ContactService(ContactRepository repository) { this.repository =
repository; }
    ...
}
```

The original in-memory behavior became one implementation of that interface (InMemoryContactRepository/InMemoryTaskRepository/InMemoryAppointmentRepository), and a second, file-backed implementation (FileContactRepository/FileTaskRepository/FileAppointmentRepository) was added, so data now actually survives the program restarting. That, in turn, made real integration tests possible for the first time — each entity now has an integration test that saves data through one service instance, reloads it through a brand-new service and repository instance pointed at the same file, and confirms the data (and later updates and deletes) genuinely persisted, rather than only existing for the life of one HashMap.

Alongside the planned enhancement, I made three smaller corrections the code review had already identified: fixed a compile-breaking, invalid-regex bug in Contact's phone validation (a regression from an earlier, working version of the same file); made the update methods on ContactService and TaskService check that an ID exists before updating it, so an unknown ID now throws a clear IllegalArgumentException instead of an unchecked NullPointerException; and standardized the Contact test classes on JUnit 5 assertions, removing a leftover JUnit 4 import that had been mixed in. I also consolidated what had been three separate Eclipse projects with no build tool into a single Maven project with proper Java packages, and added a GitHub Actions workflow that runs the full unit-and-integration suite automatically on every push — a direct, literal match for the "Automation" in this course's own name.

3. Course Outcome Coverage

In Module One, I planned for this enhancement to demonstrate Outcomes 2, 3, and 4. I believe this milestone makes substantial progress on all three, and I don't have any changes to that outcome-coverage plan:

- Outcome 2 (professional-quality communication): this narrative, and the inline comments throughout the enhanced code explaining why each change was made (not just what changed), are written to be followable by a non-technical reviewer while staying precise enough for a technical one.
- Outcome 3 (evaluating a computing solution and managing design trade-offs): choosing constructor-injected interfaces over the original hardcoded HashMap was itself a trade-off, and the file-backed repository makes a deliberate, documented trade-off of its own — it rewrites the entire file on every write, which is simple and correct at this artifact's scale but wouldn't be the right choice at a larger one. I called that trade-off out directly in the code rather than leaving it implicit.
- Outcome 4 (well-founded tools and techniques delivering a working improvement): the Maven build, the new integration tests, and the GitHub Actions CI workflow are all concrete, working tooling — not just a written recommendation — and the smaller bug fixes mean the artifact actually runs correctly now, not just in theory.

4. Reflection on the Process

The biggest challenge wasn't the repository pattern itself — it was that the artifact didn't have a real build in the first place. All three services were separate Eclipse projects with no Maven or Gradle file and

no shared package structure, so before I could add integration tests or a CI workflow, I had to consolidate everything into one properly organized project. That wasn't part of the original enhancement plan, but it turned out to be a prerequisite for it, which was a useful reminder that a plan written before touching the code doesn't always account for the state the code is actually in.

The most interesting problem I ran into was specific to the Appointment service. Appointment is immutable by design — every field, including the date, can only be set once, and the constructor rejects any date that isn't in the future at the moment of construction. That's a good rule for creating a new appointment, but it becomes a real tension for a repository that has to reconstruct an Appointment object from a stored file: if enough time has passed that a previously valid appointment's date is now in the past, reconstructing it re-runs that same future-only validation and fails. I didn't try to hide that — I documented it directly as a known limitation in FileAppointmentRepository's comments, since a production version of this would need a separate reconstruction path that trusts previously validated data instead of re-validating it. Running into that was a good, concrete example of how a validation rule that's correct for one use case (creating something new) can conflict with a different use case (reloading something that already existed) once persistence enters the picture.

I also had to decide how far to go beyond the specific plan from Module One. The code review had found more problems than just the storage coupling — the broken phone validation, the missing null checks on updates, the mixed test library imports — and none of those were explicitly part of the "repository interface" enhancement I'd scoped out. I chose to fix them anyway, since they were small, directly in the files I was already changing, and leaving a known compile-breaking bug in place for the sake of staying narrowly scoped to the plan didn't make sense. I documented each one as a distinct, separate fix in the code comments rather than folding it silently into the bigger change, so it's still clear which part of the diff is the planned enhancement and which parts are corrections.

AI Usage Acknowledgment: Generative AI (Claude) was used to help implement the repository interfaces and file-backed repositories, refactor the three service classes to use dependency injection, write the integration tests, set up the Maven project structure and GitHub Actions workflow, and draft language

for this narrative. The enhancement plan itself (from Module One), the decisions about what to fix beyond that plan, and the final review of all code and content are my own.